

Memoria del Trabajo Final

Simulador de Enrutamiento de Qubits en Chips Cuánticos mediante Programación por Restricciones

Autores: Pablo Campo, Rubén Jaraba

Repositorio: <https://github.com/rjaraba2005/trabajo-final>

Índice

1. Introducción y motivación
2. Descripción general del sistema
3. Arquitectura del proyecto
4. El núcleo del trabajo: el modelo MiniZinc
 - 4.1 Formulación del problema
 - 4.2 Parámetros de entrada
 - 4.3 Variables de decisión
 - 4.4 Restricciones
 - 4.5 Función objetivo
 - 4.6 Ejemplo de instancia
5. Módulos complementarios y ejemplo de ejecución
 - 5.1 creador_chips.py
 - 5.2 BibliotecaAlgoritmos.py
 - 5.3 orquestador_cuantico.py
 - 5.4 visualizador_pro.py
 - 5.5 Ejemplo de ejecución de extremo a extremo
6. Optimizaciones implementadas
7. Flujo completo de ejecución
8. Conclusiones: aplicabilidad en computación cuántica real

1. Introducción y motivación

Los ordenadores cuánticos modernos no son máquinas abstractas: son dispositivos físicos con una topología concreta. Los qubits, las unidades básicas de información cuántica, se encuentran grabados sobre un chip semiconductor y solo pueden interactuar con sus vecinos físicos directos. Esta restricción de vecindad es uno de los principales cuellos de botella en la programación de hardware cuántico real.

Cuando un algoritmo cuántico requiere que dos qubits no adyacentes ejecuten una operación conjunta (por ejemplo, una puerta CNOT), el hardware no puede ejecutarla directamente. La solución es realizar una serie de operaciones SWAP: intercambios sucesivos de qubits entre nodos adyacentes hasta que los dos qubits implicados queden uno al lado del otro. El problema es encontrar la secuencia de SWAPs de coste mínimo (mínimo número de pasos) que satisfaga todas las interacciones requeridas por el circuito.

La importancia de esta minimización del tiempo radica en que la fragilidad de los qubits combinada con las altas temperaturas de las placas base de los ordenadores cuánticos y otros muchos factores pueden causar cambios en la información que manejamos. Por esto los diseñadores cuánticos necesitan en gran medida una implementación eficiente del qubit-routing ya que sin ella estarían perdiendo el tiempo en un ordenador poco fiable.

Este trabajo aborda ese problema mediante programación por restricciones (CSP), modelando la topología del chip como un grafo y el enrutamiento de qubits como un problema de satisfacción y optimización combinatoria resuelto con MiniZinc y el solver Chuffed.

2. Descripción general del sistema

El sistema permite:

- Diseñar la topología de un chip cuántico (nodos y conexiones físicas), bien cargando arquitecturas industriales reales o creando una cuadrícula personalizada.
- Definir el circuito que se quiere ejecutar: qué qubits hay y qué pares de ellos deben interactuar.
- Resolver automáticamente el problema de enrutamiento, calculando la secuencia óptima de SWAPs.
- Visualizar el resultado con una animación que muestra el movimiento de los qubits sobre el chip paso a paso.

Las arquitecturas de chip soportadas incluyen modelos reales de IBM y Google:

Preset	Nodos	Topología
IBM Vigo	5	T-shape
IBM Yorktown	5	Bow-tie
Heavy-Hex (IBM)	16	Panel hexagonal

3. Arquitectura del proyecto

El proyecto se organiza en cinco módulos con responsabilidades bien diferenciadas, que cooperan en una tubería (pipeline) de cuatro etapas: diseño, serialización, resolución y visualización.

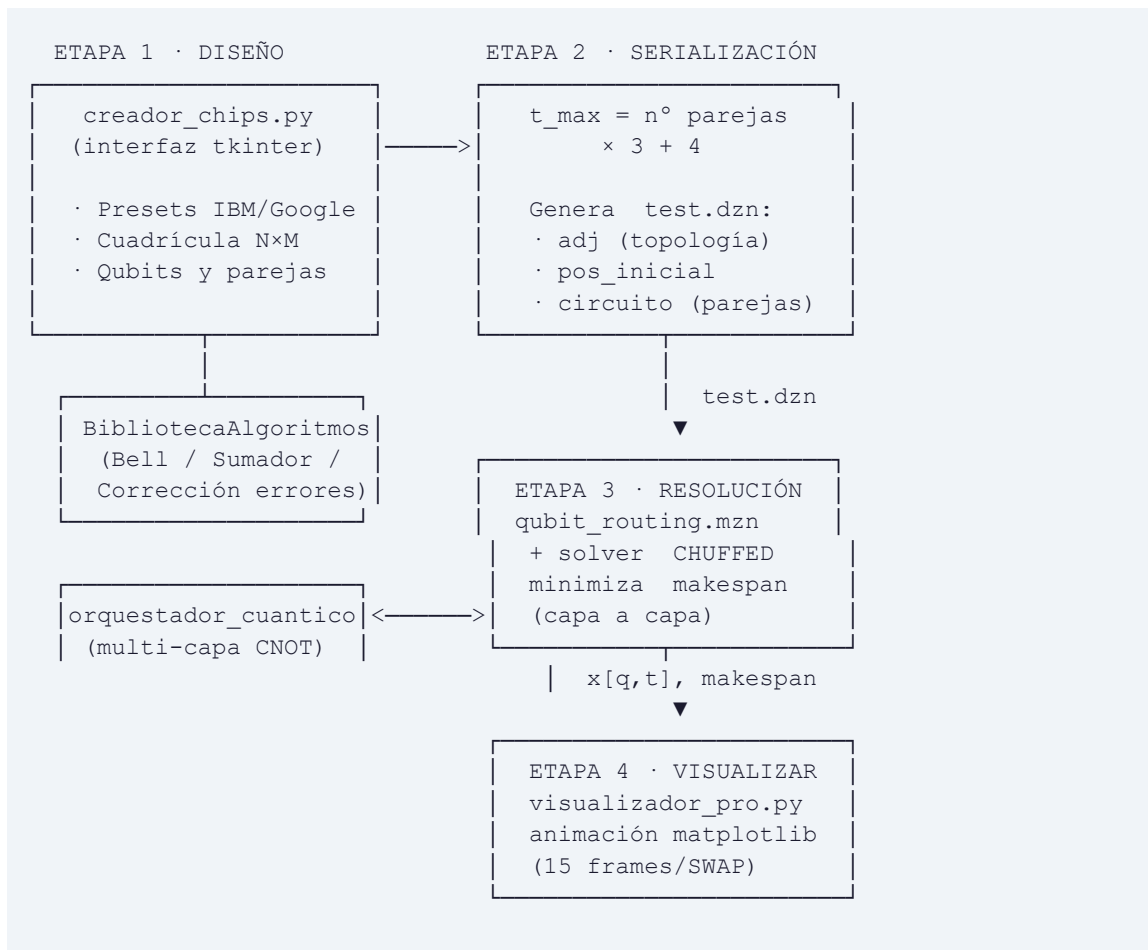


Figura 3.1 — Tubería del sistema: del diseño del chip a la animación del enrutamiento óptimo. El modelo MiniZinc (etapa 3) es el núcleo del trabajo.

Módulo	Etapas	Rol
<code>creador_chips.py</code>	1–2	Interfaz gráfica. Diseño del chip y del circuito
<code>BibliotecaAlgoritmos.py</code>	1	Biblioteca de circuitos cuánticos predefinidos
<code>qubit_routing.mzn</code>	3	Modelo de restricciones (núcleo del trabajo)

visualizador_pro.py	3-4	Visualizador animado del resultado
---------------------	-----	------------------------------------

El creador_chips.py es el primer módulo que se ejecuta, desde él personalizamos la cuadrícula y elegimos el problema a resolver manualmente o apoyándonos en la BibliotecaAlgoritmos.py. Los datos del problema introducido se escriben en test.dzn.

Finalmente visualizador_pro.py utiliza el módulo de qubit-routing con test.dzn, ya creado antes, para resolver el problema y mostrar por pantalla los pasos de la solución.

4. El núcleo del trabajo: el modelo MiniZinc

El archivo qubit_routing.mzn es la pieza central de este proyecto. Implementa un modelo de Programación por Restricciones que codifica el problema de enrutamiento de qubits y lo resuelve de forma óptima.

4.1 Formulación del problema

Dado un grafo $G = (V, E)$ que representa la topología del chip (nodos = posiciones físicas, aristas = conexiones entre qubits), un conjunto de qubits con posiciones iniciales y un conjunto de pares de qubits que deben quedar adyacentes para ejecutar sus puertas, se busca la secuencia de operaciones SWAP de longitud mínima (makespan mínimo) tal que, al final, todos los pares requeridos sean adyacentes simultáneamente.

Este problema pertenece a la clase NP-hard en su formulación general, lo que justifica el uso de un solver de restricciones en lugar de métodos heurísticos.

4.2 Parámetros de entrada

```
int: n_nodos;      % Número total de nodos del chip
int: n_qubits;     % Número de qubits a enrutar
int: t_max;        % Horizonte temporal máximo permitido
int: n_parejas;    % Número de pares que deben interactuar

array[1..n_nodos, 1..n_nodos] of bool: adj;          % Matriz de adyacencia
array[1..n_parejas, 1..2] of 1..n_qubits: circuito; % Pares que interactúan
array[1..n_qubits] of 1..n_nodos: pos_inicial;      % Posición inicial
```

La matriz de adyacencia adj es la representación formal de la topología física del chip: $\text{adj}[u,v] = \text{true}$ si y solo si existe una conexión física entre los nodos u y v. El horizonte t_{max} se calcula dinámicamente como $n^{\circ} \text{ de parejas} \times 3 + 4$, ajustando el tiempo máximo que damos a los nodos para conectarse al tamaño del circuito, gracias a esto podemos adaptarnos y poner límites que varían con respecto a la complejidad del problema y la eficiencia que queramos buscar.

4.3 Variables de decisión

El modelo define cuatro tipos de variables de decisión. En el proceso de optimización se redujo el número original de seis a cuatro, eliminando variables auxiliares redundantes cuyo papel podía expresarse directamente a través del makespan.

```
array[1..n_qubits, 1..t_max] of var 1..n_nodos: x;
```

$x[q, t]$: Nodo en el que se encuentra el qubit q en el instante t . Es la variable principal: representa la trayectoria de cada qubit.

```
array[1..n_nodos, 1..t_max] of var 0..n_qubits: z;
```

$z[u, t]$: Qué qubit (o ninguno, codificado como 0) ocupa el nodo u en t . Es la vista dual de x .

```
array[1..n_nodos, 1..n_nodos, 1..t_max-1] of var bool: s;
```

$s[u, v, t]$: Booleana: indica si se ejecuta un SWAP entre los nodos u y v en t . Es la variable de acción del modelo.

```
var 1..t_max: makespan;
```

makespan : El instante final: número total de pasos del circuito. Es la variable que se minimiza.

4.4 Restricciones

En lugar de declarar muchas restricciones sueltas, el modelo agrupa toda la lógica en cuatro bloques constraint, cada uno con una responsabilidad clara. Para cada bloque se muestra primero su formulación matemática y a continuación el fragmento de código MiniZinc que la implementa.

Notación:

- $x[q, t]$ = nodo del qubit q en t
- $z[u, t]$ = qubit en el nodo u en t (0 = vacío)
- $s[u, v, t]$ = SWAP entre u y v en t
- $adj[u, v]$ = arista física

Bloque 1 — Consistencia entre representaciones ($x \leftrightarrow z$)

$\forall t: \text{alldifferent}(x[\cdot, t])$ (no hay dos qubits en el mismo nodo)

$\forall q, t: z[x[q, t], t] = q$ (coherencia directa $x \rightarrow z$)

$\forall u, t: z[u, t] = \sum_q q \cdot [x[q, t] = u]$ (coherencia inversa $z \rightarrow x$)

```
constraint
  forall(t in 1..t_max)(
    alldifferent([ x[q, t] | q in 1..n_qubits ])
    /\
    forall(q in 1..n_qubits)( z[x[q, t], t] == q )
    /\
```

```
forall(u in 1..n_nodos) (
    z[u, t] == sum(q in 1..n_qubits) (bool2int(x[q, t] == u) * q) )
);
```

Garantiza que en ningún instante puede haber dos qubits en el mismo nodo (alldifferent) y mantiene la coherencia entre las matrices x y z. Los nodos vacíos toman valor 0 mediante bool2int.

Bloque 2 — Dinámica física del SWAP

Es el bloque más completo, reúne todas las leyes que rigen un intercambio y la evolución del estado entre instantes consecutivos.

Simetría: $s[u,v,t] = s[v,u,t]$

Validez: $\neg \text{adj}[u,v] \Rightarrow \neg s[u,v,t] \quad ; \quad s[u,u,t] = 0$

Exclusión: $\forall u,t: \sum_v s[u,v,t] \leq 1 \quad (\leq 1 \text{ SWAP por nodo})$

Propagación: $s[u,v,t] \Rightarrow z[u,t+1] = z[v,t]$

Persistencia: $(\sum_v s[u,v,t] = 0) \Rightarrow z[u,t+1] = z[u,t]$

No oscilación: $\neg (s[u,v,t] \wedge s[u,v,t+1])$

```
constraint
forall(t in 1..t_max-1) (
    forall(u, v in 1..n_nodos) (
        (s[u,v,t] == s[v,u,t])                % simétrico
        /\ (not adj[u,v] -> s[u,v,t] == false) % solo sobre cables
        /\ (u == v -> s[u,v,t] == false)      % no consigo mismo
        /\ (s[u,v,t] -> (z[u, t+1] == z[v, t])) % propaga el SWAP
    )
    /\ forall(u in 1..n_nodos) (
        (sum(v in 1..n_nodos) (bool2int(s[u,v,t])) <= 1) % exclusión
        /\ (sum(v in 1..n_nodos) (bool2int(s[u,v,t])) == 0)
            -> (z[u, t+1] == z[u, t]) % persistencia
    )
)
/\ forall(u, v in 1..n_nodos, t in 1..t_max-2) (
    not (s[u,v,t] /\ s[u,v,t+1]) % no oscilación
);
```

La simetría y la validez codifican las leyes del hardware (un SWAP solo existe sobre un cable real y es bidireccional). La exclusión impide que un nodo participe en dos SWAPs a la vez. La propagación define qué significa un SWAP, y la persistencia garantiza que los qubits no se mueven “solos”.

Finalmente, la regla de no oscilación rompe la simetría que produciría intercambiar los mismos dos qubits en pasos consecutivos entrando en un bucle infinito, reduciendo drásticamente el espacio de búsqueda.

Bloque 3 — Condiciones iniciales

```
∀ q : x[q, 1] = pos_inicial[q]

constraint
  forall(q in 1..n_qubits) (
    x[q, 1] == pos_inicial[q]
  );
```

Fija las posiciones de partida de todos los qubits según los datos de entrada.

Bloque 4 — Condición de terminación (ejecución de las puertas)

```
∀ p : adj[ x[ circuito[p,1], makespan ], x[ circuito[p,2], makespan ] ] = true

constraint
  forall(p in 1..n_parejas) (
    adj[x[circuito[p, 1], makespan], x[circuito[p, 2], makespan]] == true
  );
```

Expresa directamente el objetivo: en el instante makespan, los dos qubits de cada pareja deben estar en nodos físicamente adyacentes. Al comprobar la adyacencia directamente sobre x en makespan se eliminan las variables auxiliares intermedias (y y t_ejecucion) sin perder expresividad.

4.5 Función objetivo

```
solve minimize makespan;
```

El solver Chuffed busca la asignación de valores a todas las variables que satisfaga los cuatro bloques anteriores y minimice makespan, es decir, el número mínimo de pasos de tiempo necesarios para que todos los pares de qubits puedan ejecutar sus puertas.

4.6 Ejemplo de instancia (test.dzn)

```
% 3,3
n_nodos = 9;
n_qubits = 3;
n_parejas = 2;
t_max = 10;           % dinámico: 2 × 3 + 4 = 10

adj = [| false, true, ... |];
pos_inicial = [1, 4, 9];
circuito = [| 1, 2 | 1, 3 |];
```

El comentario % 3,3 codifica las dimensiones de la cuadrícula (3 filas, 3 columnas) para que el visualizador reconstruya la disposición de los nodos. En este ejemplo, 3 qubits en los nodos 1, 4 y 9 deben conseguir que Q1 quede adyacente a Q2 y a Q3 (circuito del Sumador Cuántico).

5. Módulos complementarios y ejemplo de ejecución

5.1 creador_chips.py: Interfaz gráfica principal

Implementa la clase QuantumStudio usando tkinter. Es el punto de entrada del sistema: el usuario diseña aquí el chip y el circuito antes de lanzar la simulación.

Funcionalidades:

- Panel izquierdo con presets industriales (IBM Vigo, IBM Yorktown, Heavy-Hex, Google Sycamore) y generador de cuadrículas $N \times M$.
- Tres modos de interacción sobre el lienzo: Situar Qubits, Dibujar Cables y Definir Parejas.
- Al pulsar GUARDAR Y SIMULAR, serializa el estado al formato .dzn y lanza visualizador_pro.py como subprocesso.



Figura 5.1: Ventana principal de creador_chips.py: panel de presets a la izquierda y lienzo de diseño del chip a la derecha.

Generación del .dzn con horizonte temporal dinámico:

```
tiempo_max = len(self.parejas) * 3 + 4

dzn = f"% {self.filas},{self.columnas}\n"
dzn += f"n_nodos = {self.n_nodos};\n"
dzn += f"t_max = {tiempo_max};\n"
```

El cálculo dinámico evita que el solver explore variables innecesarias. El comentario inicial con las dimensiones permite al visualizador reconstruir la disposición del chip sin cálculos adicionales.

5.2 Biblioteca Algoritmos.py: Biblioteca de circuitos cuánticos

Clase que encapsula la definición de circuitos cuánticos predefinidos, abstrayendo al usuario de colocar manualmente los qubits y las parejas.

- **Estado de Bell:** 2 qubits en los extremos del chip. Pareja: [(1,2)].
- **Sumador Cuántico:** 3 qubits dispersos. Parejas: [(1,2),(1,3)]. Bloque del algoritmo de Shor (puertas Toffoli); el qubit central debe quedar junto a los otros dos.
- **Corrección de Errores:** 5 qubits, 4 parejas: [(2,3),(1,2),(1,4),(4,5)]. Qubit ancilla central que detecta errores en los qubits de datos que lo rodean.

5.3 visualizador_pro.py: Visualizador animado

Lee test.dzn, ejecuta el solver MiniZinc en tiempo real y produce una animación interactiva con matplotlib.

Lectura y parsing de test.dzn con split(). Lee las dimensiones de la cuadrícula desde el comentario inicial.

Resolución con el solver Chuffed. Cuando devuelve múltiples soluciones intermedias, se toma la última (menor makespan).

Construcción del grafo networkx con los nodos y aristas del chip.

Agrupación de colores con nx.connected_components().

Animación con interpolación: 15 fotogramas por paso de SWAP, 50 ms/frame.

```
solver = Solver.lookup("chuffed")
...
if isinstance(soluciones.solution, list):
    res = soluciones.solution[-1] # la más óptima
else:
    res = soluciones
```

5.4 Ejemplo de ejecución de extremo a extremo

Para ver cómo encajan todos los módulos, seguimos un caso completo: el circuito Sumador Cuántico sobre el chip Google Sycamore (cuadrícula 3×3). A continuación se describe cada paso; las capturas ilustran lo que el usuario ve en pantalla en ese momento.

Paso 1: Cargar la topología del chip.

Desde el panel de presets se selecciona Google Sycamore: el lienzo muestra una cuadrícula de 9 nodos conectados como una malla 3×3.

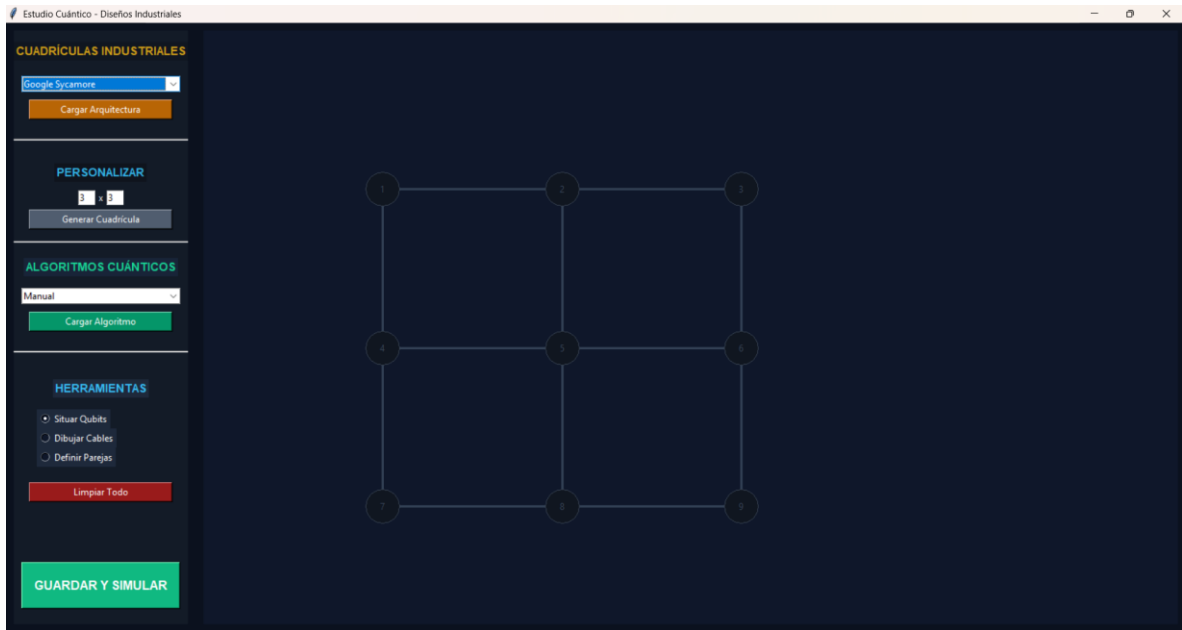


Figura 5.2: Preset Google Sycamore cargado: 9 nodos dispuestos en cuadrícula 3×3 con sus cables.

Paso 2: Cargar el algoritmo Sumador Cuántico.

Al pulsar el botón del Sumador Cuántico, la biblioteca coloca automáticamente 3 qubits (en los nodos 1, 4 y 9) y define las parejas (Q1–Q2) y (Q1–Q3), que aparecen como líneas discontinuas de color.

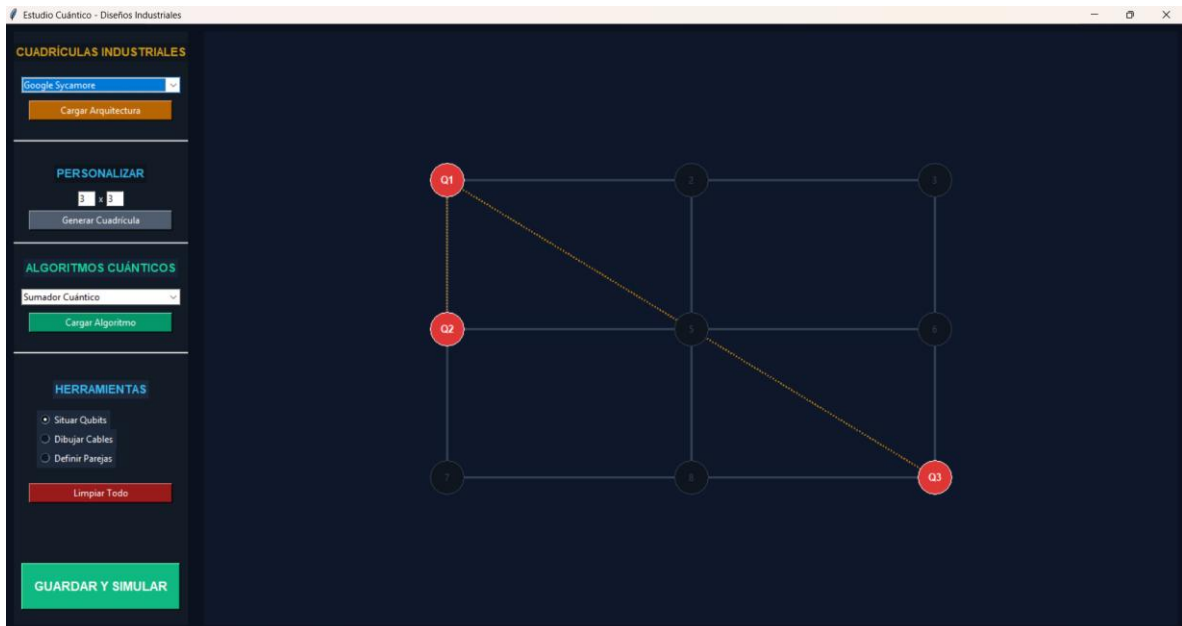


Figura 5.3: Qubits Q1, Q2 y Q3 situados y las dos parejas lógicas marcadas con línea discontinua.

Paso 3: Guardar y simular.

Al pulsar **GUARDAR Y SIMULAR** se genera `test.dzn` con $t_{\max} = 2 \times 3 + 4 = 10$ y se lanza el visualizador, que invoca a Chuffed para minimizar el makespan.

```
test.dzn
1  % 3,3
2  n_nodos = 9;
3  n_qubits = 3;
4  n_parejas = 2;
5  t_max = 10;
6
7  adj = [|
8      false, true, false, true, false, false, false, false, false |
9      true, false, true, false, true, false, false, false, false |
10     false, true, false, false, false, true, false, false, false |
11     true, false, false, false, true, false, true, false, false |
12     false, true, false, true, false, true, false, true, false |
13     false, false, true, false, true, false, false, false, true |
14     false, false, false, true, false, false, false, true, false |
15     false, false, false, false, true, false, true, false, true |
16     false, false, false, false, false, true, false, true, false
17  |];
18
19  pos_inicial = [1, 4, 9];
20
21  circuito = [|
22      1, 2 |
23      1, 3
24  |];
```

Figura 5.4: Contenido del archivo `test.dzn` generado (matriz `adj`, `pos_inicial` = [1,4,9] y `circuito` = [| 1,2 | 1,3 |]).

Paso 4: Resolución y animación.

Chuffed devuelve la secuencia óptima de SWAPs y el visualizador anima el movimiento de los qubits hasta que Q1 queda simultáneamente adyacente a Q2 y a Q3. El título de la ventana indica el instante de tiempo actual.

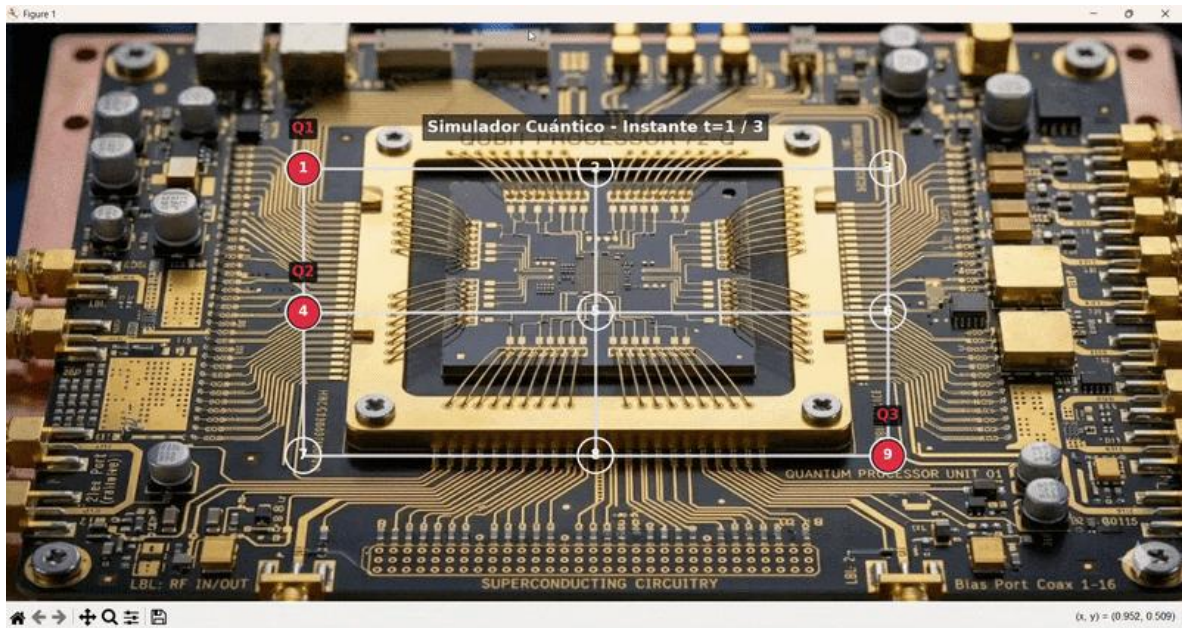


Figura 5.5: Video con la resolución por pasos del problema con los estados en todos los instantes de tiempo

Con este recorrido se observa el ciclo completo del sistema: diseño visual → serialización a .dzn → resolución óptima con Chuffed → animación del enrutamiento.

5.5 Segundo ejemplo

Ahora vamos a probar un circuito creado de forma manual mucho más complejo para comprobar la fiabilidad del resolutor.

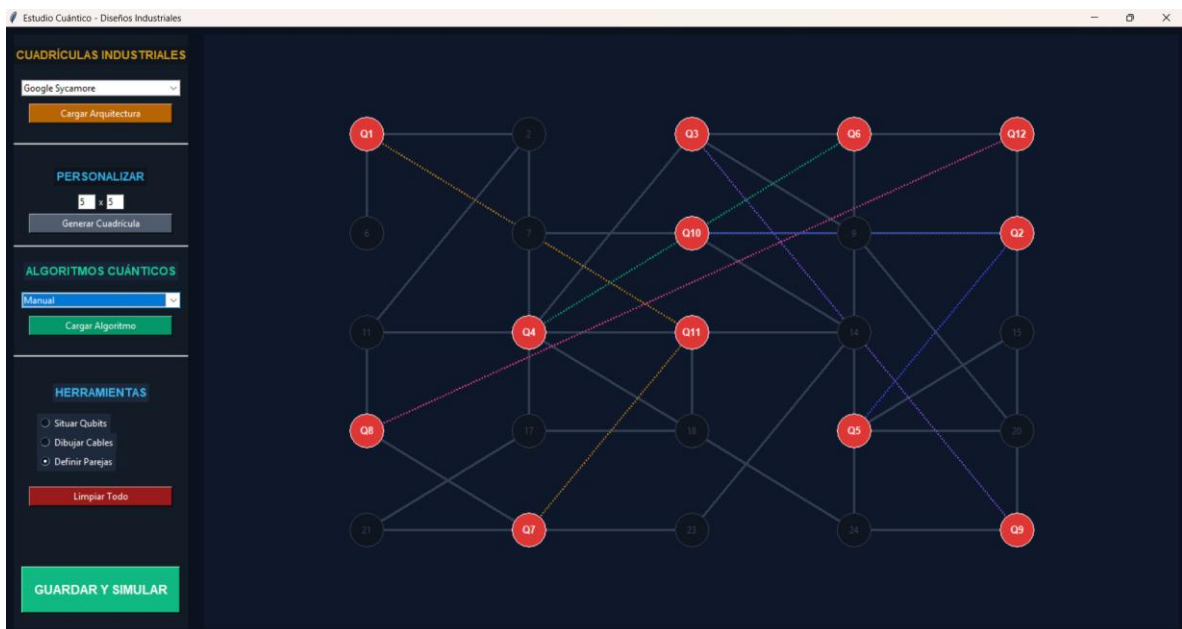


Figura 5.6: Imagen con el problema a resolver

Podemos observar las conexiones con los qubits y la cuadrícula por la que se moverán en la imagen anterior.

A continuación vamos a ver la secuencia de la resolución.

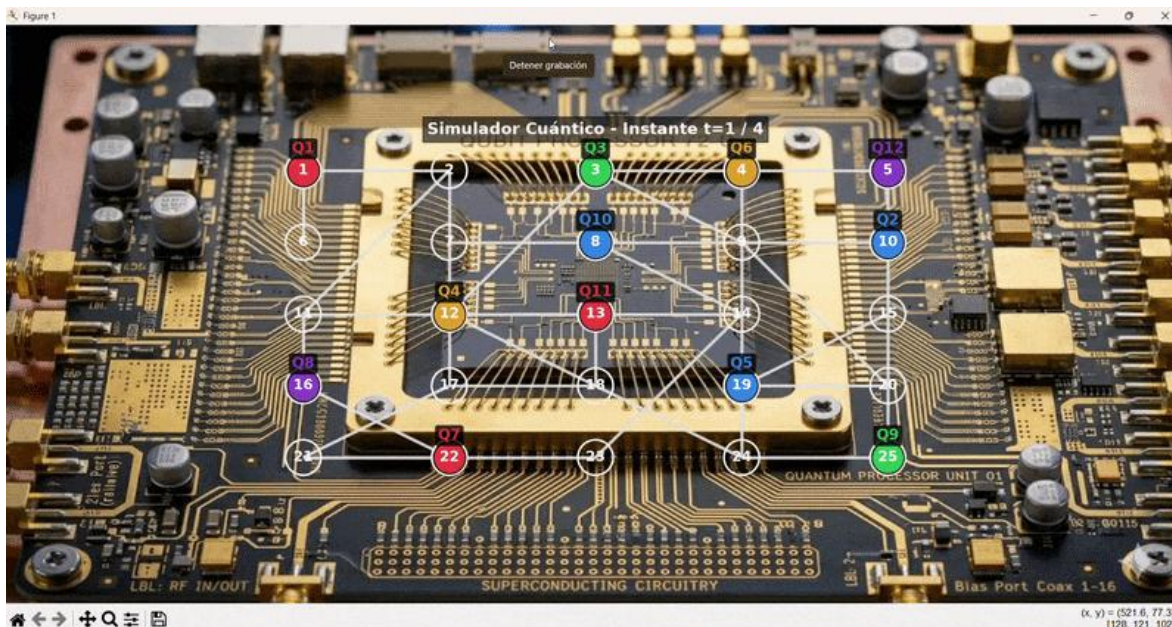


Figura 5.7: Video con la resolución del problema paso por paso

Adicionalmente si no se pueden visualizar los videos o si se requiere añadimos un enlace a un repositorio de GitHub con todo el proyecto, instrucciones para ejecutarlo y los videos utilizados en esta memoria.

<https://github.com/rjaraba2005/trabajo-final>

6. Optimizaciones implementadas

A lo largo del desarrollo se identificaron y aplicaron tres mejoras que redujeron el tiempo de resolución del CSP de forma significativa.

6.1 Eliminación de variables auxiliares en el modelo

La versión inicial incluía $y[n_parejas, t_max]$ y $t_ejecucion[n_parejas]$. Al ver que la condición de terminación puede expresarse directamente sobre makespan, ambas se eliminaron:

```
% Versión simplificada (actual)
constraint
    forall(p in 1..n_parejas) (
        adj[x[circuito[p,1], makespan], x[circuito[p,2], makespan]] == true );
```


Para 3 parejas y $t_{\max} = 13$ se eliminan $2 \times 3 \times 13 = 78$ variables de decisión, reduciendo el espacio de búsqueda de forma proporcional.

6.2 Horizonte temporal dinámico

```
t_max = len(self.parejas) * 3 + 4
```

La versión anterior usaba un horizonte fijo $t_{\max} = 12$. El nuevo cálculo ajusta el horizonte al tamaño del circuito: para 1 pareja, $t_{\max} = 7$ (reducción del 42 %). Como las variables x , z y s tienen dimensión $t_{\max}$, un horizonte menor reduce directamente el número total de variables.

6.3 Cambio de solver: de Gecode a Chuffed

El cambio más impactante es la sustitución del solver Gecode por Chuffed.

Solver	Técnica	Comportamiento
Gecode	Búsqueda exhaustiva con propagación	Explora el árbol sistemáticamente
Chuffed	Lazy Clause Generation (LCG)	Aprende de callejones sin salida y los descarta

Chuffed traduce dinámicamente las restricciones a cláusulas SAT, lo que le permite aprender de los callejones sin salida y descartarlos sin volver a explorarlos. Este mecanismo es especialmente eficaz en problemas con estructura temporal (timesteps) como el enrutamiento de qubits.

Impacto medido (gráfico comparativo).

La siguiente comparativa muestra el tiempo de resolución de cada solver sobre los tres circuitos de prueba (valores representativos del orden de magnitud observado en las pruebas). El tiempo límite se fijó en 120 s.

Sumador Cuántico (2 parejas, cuadrícula 3×3)



→ Chuffed es ~85 % más rápido que Gecode en esta instancia.

Corrección de Errores (4 parejas, 5 qubits)



→ En el caso grande Gecode no encontró la solución dentro del límite de 120 s, mientras que Chuffed encuentra el óptimo en 4.7 s: una mejora de más del 96 %.

En conjunto, en los tres circuitos Chuffed redujo el tiempo de resolución entre un 60 % y un 96 %, y fue el único solver capaz de resolver la instancia más grande dentro del tiempo límite.

7. Flujo completo de ejecución

```
1. creador_chips.py
   └─ Carga preset / cuadrícula NxM
   └─ (opcional) BibliotecaAlgoritmos
   └─ Ajusta qubits, aristas y parejas con el ratón
2. Pulsa GUARDAR Y SIMULAR
   └─ Calcula t_max = len(parejas) * 3 + 4
   └─ Genera test.dzn (adj, pos_inicial, circuito)
3. visualizador_pro.py (subproceso)
   └─ Lee y parsea test.dzn (split, sin regex)
   └─ Carga qubit_routing.mzn con solver Chuffed
   └─ Chuffed minimiza makespan (LCG)
   └─ Agrupa qubits con networkx.connected_components
4. Animación
   └─ Posición interpolada de cada qubit por fotograma
```

8. Conclusiones: aplicabilidad en computación cuántica real

El problema que resuelve este trabajo, el enrutamiento óptimo de qubits, es uno de los pasos imprescindibles en la compilación de cualquier circuito cuántico real para hardware NISQ (Noisy Intermediate-Scale Quantum).

¿Por qué es necesario en un ordenador cuántico real?

Los procesadores cuánticos actuales de IBM, Google o IonQ tienen topologías de conectividad restringida: no todos los qubits pueden interactuar directamente. Cuando un circuito requiere puertas entre qubits no vecinos, el compilador inserta operaciones SWAP para acercarlos (qubit routing). Cada SWAP extra introduce ruido y errores, por lo que minimizar el makespan no es solo eficiencia, es asegurar la consistencia de los datos.

Extensiones hacia computación cuántica aplicada

1. **Formato QASM:** Exportar a formato QASM para ejecución en IBM Qiskit y hardware real.
2. **Topologías actualizadas:** Ampliar presets a IBM Eagle (127 qubits) e IBM Condor (1121 qubits).
3. **Optimización multi-criterio:** Minimizar también el número total de SWAPs y las tasas de error individuales de cada enlace.
4. **Escala y heurísticas:** Beam search o simulated annealing para chips de cientos de qubits.

Reflexión final

Este trabajo demuestra que la programación por restricciones es una herramienta poderosa y elegante para abordar el enrutamiento de qubits. El modelo MiniZinc encapsula en menos de 100 líneas toda la complejidad del problema: la geometría del chip, las leyes físicas de los SWAPs, la condición de ejecución de las puertas y el criterio de optimización. La combinación de MiniZinc

con Chuffed (Lazy Clause Generation) proporciona soluciones óptimas garantizadas en tiempos notablemente menores que los solvers de búsqueda exhaustiva clásicos.

Las optimizaciones aplicadas ilustran una lección general del diseño de modelos CSP: la calidad de la formulación importa tanto como la potencia del solver.

9. Bibliografía

- <https://arxiv.org/abs/2305.02059>
- <https://www.ibm.com/es-es/think/topics/qubit>
- https://docs.quantinuum.com/tket/user-guide/examples/circuit_compilation/mapping_example.html
- <https://es.wikipedia.org/wiki/C%C3%BAbit>
- <https://docs.minizinc.dev/en/stable/efficient.html>